

Python standard library and OOP

Useful modules for quick implementations
&&

Object Oriented Programming implementation in Python

Isn't Python alone powerful enough, you poser ?

- Tons of standard modules included in the base Python installation
- You can use them on training websites and during contests
- Many tools implementing commonly functions commonly used in competitive programming



Hold on, there's a fuckton of them, which ones should I try ?

BIG ONES

- itertools : operations on iterators, generate combinations...
- math : math
- heapq : Implementation of a priority queue
- collections : data structures
- bisect : Array bisection algorithm

Itertools (not gonna lie, it's mainly for brute-forcing)

- Main goal: to create effective iterators for you to work on
- Itertools is optimized for the creation of iterators
- Will allow you to gain time if you know what you are looking for
- Really useful to exploring basic combinatorics spaces

Combinatoric iterators:

Iterator	Arguments	Results
<code>product()</code>	<code>p, q, ...</code> <code>[repeat=1]</code>	cartesian product, equivalent to a nested for-loop
<code>permutations()</code>	<code>p[, r]</code>	r-length tuples, all possible orderings, no repeated elements
<code>combinations()</code>	<code>p, r</code>	r-length tuples, in sorted order, no repeated elements
<code>combinations_with_replacement()</code>	<code>p, r</code>	r-length tuples, in sorted order, with repeated elements

Examples	Results
<code>product('ABCD', repeat=2)</code>	AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD
<code>permutations('ABCD', 2)</code>	AB AC AD BA BC BD CA CB CD DA DB DC
<code>combinations('ABCD', 2)</code>	AB AC AD BC BD CD
<code>combinations_with_replacement('ABCD', 2)</code>	AA AB AC AD BB BC BD CC CD DD

Itertools - Other useful functions

Infinite iterators:

Iterator	Arguments	Results	Example
<code>count()</code>	start, [step]	start, start+step, start+2*step, ...	<code>count(10) --> 10 11 12 13 14 ...</code>
<code>cycle()</code>	p	p0, p1, ... plast, p0, p1, ...	<code>cycle('ABCD') --> A B C D A B C D ...</code>
<code>repeat()</code>	elem [,n]	elem, elem, elem, ... endlessly or up to n times	<code>repeat(10, 3) --> 10 10 10</code>

To infinity and beyond

Iterator	Arguments	Results	Example
<code>accumulate()</code>	p [,func]	p0, p0+p1, p0+p1+p2, ...	<code>accumulate([1,2,3,4,5]) --> 1 3 6 10 15</code>
<code>compress()</code>	data, selectors	(d[0] if s[0]), (d[1] if s[1]), ...	<code>compress('ABCDEF', [1,0,1,0,1,1]) --> A C E F</code>
<code>dropwhile()</code>	pred, seq	seq[n], seq[n+1], starting when pred fails	<code>dropwhile(lambda x: x<5, [1,4,6,4,1]) --> 6 4 1</code>
<code>filterfalse()</code>	pred, seq	elements of seq where pred(elem) is false	<code>filterfalse(lambda x: x%2, range(10)) --> 0 2 4 6 8</code>
<code>takewhile()</code>	pred, seq	seq[0], seq[1], until pred fails	<code>takewhile(lambda x: x<5, [1,4,6,4,1]) --> 1 4</code>

Who knows... it might be useful

Math, Some situational Functions...

- `math.comb`(n, k)
- `math.factorial`(x)
- `math.gcd`(**integers*) only two arguments were supported before 3.9
- `math.trunc`(x)
- `math.exp`(x)
- `math.log`(x [, *base*])
- `math.cos`(x)
- `math.dist`(p, q)
- `math.degrees`(x) / `math.radians`(x)
- `pow()` # The one NOT in the math library has a modulus parameter

...and some Constants

- `math.pi`
- `math.e`
- `math.inf` equivalent to `float('inf')`

π : 3.141592653589793
e: 2.7182818284590452
Engineers:



Heapq : a priority queue

Will be essential for Dijkstra's Algorithm $\mathcal{V}(\mathcal{E}, \mathcal{V})$

```
from heapq import *  
  
list = [5, 2, 4, 1, 3, 2]  
heapify(list)  
  
print(heapop(list))  
heappush(list, 5)  
print(heapop(list))  
print(list)
```

Collections : data structures, containers

Deque : implementation of a 2-ended-queue

Can be a stack, as well as a queue : append left, right, push left, right

```
from collections import deque
a = deque([1, 3])
a.append(5)    # Append right
a.appendleft(2)
print(a.pop()) # Pop right
print(a.popleft())
print(a.popleft())
print(a)
```


Collections : Counter

Count occurrences in lists

Many easy exercises are based on counting => Cheese them !

```
from collections import Counter
l = [2, 2, 1, 5, 2, 5, 1, 1]
print(Counter(l))    # {2: 3, 1: 3, 5:
2}
print(Counter(l).most_common())
```

Collections : defaultdict

Dicts with elements initialized as a list, an object...

```
from collections import defaultdict
d = defaultdict(list)
d[1].append(2)
print(d.items())
```

Bisect : array binary search

Who said you have to implement your own binary search algorithm ?

Given a sorted list, get the position of an element to be inserted

```
from bisect import bisect_left

data = [6, 4, 2, 7, 2, 8, 9]
data.sort()
print(data)
print(a := bisect_left(data, 5))
data.insert(a, 5)
print(data)
```

But wait, there's more !

smol ones

- copy : hard copy an objects, because you don't want to copy the pointer
- functools : magic functions, like automatic memoization :O
- re : Regular expressions for pattern matching
- json : you guessed it
- pprint : print, but prettier
- fractions : fraction
- base64 : base64

Les paradigmes de programmation

Imperative:



Object-Oriented:



Functional:



Beaucoup d'options...

- Imperative
 - procedural
 - **Object-Oriented**
- Declarative
 - Functional
 - Logic
 - Reactive
- Concurrent
- Constrained
- Distributed
- Generic
- Metaprogramming

- **Python**: What if everything was a dict?
- **Java**: What if everything was an object?
- **JavaScript**: What if everything was a dict *and* an object?
- **C**: What if everything was a pointer?
- **APL**: What if everything was an array?
- **Tcl**: What if everything was a string?
- **Prolog**: What if everything was a term?
- **LISP**: What if everything was a pair?
- **Scheme**: What if everything was a function?
- **Haskell**: What if everything was a monad?
- **Assembly**: What if everything was a register?
- **Coq**: What if everything was a type/proposition?
- **COBOL**: WHAT IF EVERYTHING WAS UPPERCASE?
- **C#**: What if everything was like Java, but different?
- **Ruby**: What if everything was monkey patched?
- **Pascal**: BEGIN What if everything was structured? END
- **C++**: What if we added everything to the language?
- **C++11**: What if we forgot to stop adding stuff?
- **Rust**: What if garbage collection didn't exist?
- **Go**: What if we tried designing C a second time?
- **Perl**: What if shell, sed, and awk were one language?
- **Perl6**: What if we took the joke too far?
- **PHP**: What if we wanted to make SQL injection easier?
- **VB**: What if we wanted to allow anyone to program?
- **VB.NET**: What if we wanted to stop them again?
- **Forth**: What if everything was a stack?
- **ColorForth**: What if the stack was green?
- **PostScript**: What if everything was printed at 600dpi?
- **XSLT**: What if everything was an XML element?
- **Make**: What if everything was a dependency?
- **m4**: What if everything was incomprehensibly quoted?
- **Scala**: What if Haskell ran on the JVM?
- **Clojure**: What if LISP ran on the JVM?
- **Lua**: What if game developers got tired of C++?
- **Mathematica**: What if Stephen Wolfram invented everything?
- **Malbolge**: What if there is no god?

Pourquoi la POO ?

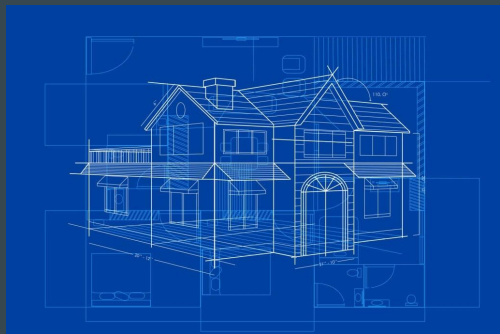
- Réunir données et traitement
- Code plus générique
- Abstraction des traitements
- *Rendre le code plus lisible (....)*
- Conteneuriser, chaque bloc de code à un but précis



Les briques de base : l'objet et la classe (😎)

La classe

C'est ce qu'on définit, c'est là qu'on décrit à quoi ressemble un objet, quels seront ses **attributs** et ses **méthodes**



L'objet

C'est ce qu'on manipule dans le programme, on sait ce qu'il contient et comment l'utiliser grâce à la **classe** qui définit son comportement



Qu'est-ce qui se passe donc ?

```
class Point:
    def __init__(self, px, py):
        self.x = px
        self.y = py

    def afficher(self):
        print(self.x, self.y)
```

```
point = Point(1, 2)
point.afficher() # 1 2
```

x : attribut
y : attribut
afficher : méthode
__init__ : méthode spéciale
(constructeur)

self désigne l'objet depuis lequel la méthode est appelée

point.afficher() -> dans afficher, self est l'objet point (un pointeur sur ses données).

Point est la classe, point est un objet de la classe Point

Le CONSTRUCTEUR (important)

Le constructeur est une méthode spéciale, elle est appelée à la création d'un objet.

On peut lui passer des paramètres comme n'importe quelle méthode.

Permet d'initialiser les attributs de notre objet selon les valeurs passées en paramètre et renvoie donc une **instance** de cet objet.

En python : `__init__`



La surcharge d'opérateurs

Pour utiliser des opérateurs tels que +, *, [],... sur vos objets.
On définit une méthode spéciale. Exemple pour le + :

```
class Point:
    def __init__(self, px, py):
        self.x = px
        self.y = py

    def afficher(self):
        print(self.x, self.y)

    def __add__(self, point2):
        self.x += point2.x
        self.y += point2.y

p1 = Point(1, 2)
p2 = Point(2, 3)

p1 + p2
# Equivalent : p1.__add__(p2)

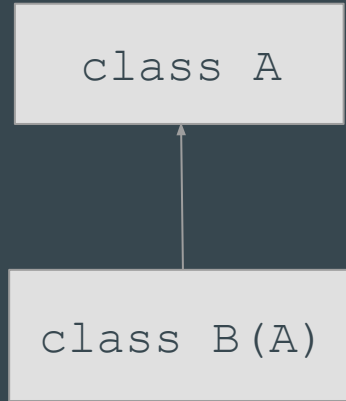
p1.afficher() # 3 5
```

La surcharge d'opérateurs

Magic Method	When it gets invoked (example)	Explanation
<code>__new__(cls [,...])</code>	<code>instance = MyClass(arg1, arg2)</code>	<code>__new__</code> is called on instance creation
<code>__init__(self [,...])</code>	<code>instance = MyClass(arg1, arg2)</code>	<code>__init__</code> is called on instance creation
<code>__cmp__(self, other)</code>	<code>self == other</code> , <code>self > other</code> , etc.	Called for any comparison
<code>__pos__(self)</code>	<code>+self</code>	Unary plus sign
<code>__neg__(self)</code>	<code>-self</code>	Unary minus sign
<code>__invert__(self)</code>	<code>~self</code>	Bitwise inversion
<code>__index__(self)</code>	<code>x[self]</code>	Conversion when object is used as index
<code>__nonzero__(self)</code>	<code>bool(self)</code>	Boolean value of the object
<code>__getattr__(self, name)</code>	<code>self.name</code> # name doesn't exist	Accessing nonexistent attribute
<code>__setattr__(self, name, val)</code>	<code>self.name = val</code>	Assigning to an attribute
<code>__delattr__(self, name)</code>	<code>del self.name</code>	Deleting an attribute
<code>__getattribute__(self, name)</code>	<code>self.name</code>	Accessing any attribute
<code>__getitem__(self, key)</code>	<code>self[key]</code>	Accessing an item using an index
<code>__setitem__(self, key, val)</code>	<code>self[key] = val</code>	Assigning to an item using an index
<code>__delitem__(self, key)</code>	<code>del self[key]</code>	Deleting an item using an index
<code>__iter__(self)</code>	<code>for x in self</code>	Iteration
<code>__contains__(self, value)</code>	<code>value in self</code> , <code>value not in self</code>	Membership tests using <code>in</code>
<code>__call__(self [,...])</code>	<code>self(args)</code>	"Calling" an instance
<code>__enter__(self)</code>	<code>with self as x:</code>	<code>with</code> statement context managers
<code>__exit__(self, exc, val, trace)</code>	<code>with self as x:</code>	<code>with</code> statement context managers
<code>__getstate__(self)</code>	<code>pickle.dump(pk1_file, self)</code>	Pickling
<code>__setstate__(self)</code>	<code>data = pickle.load(pk1_file)</code>	Pickling

Liste pas forcément à jour et non exhaustives des “méthodes magiques” de Python

L'héritage : étendre les fonctionnalités d'une classe



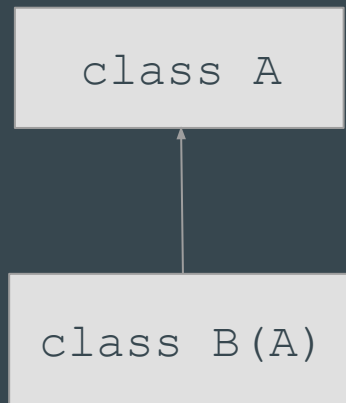
Si la classe A possède des attributs et des méthodes, la classe B les possédera aussi car elle les **héríte** de A.

Ce mécanisme permet d'étendre les fonctionnalités d'une classe, ou de fournir une spécialisation d'une classe plus générale.

Permet de construire votre propre graphe d'héritage avec vos classes pour réduire le code dupliqué.

Ou permet de modifier le comportement d'une classe préexistante dans le langage.

L'héritage : la surcharge de méthodes



Si la classe A possède des attributs et des méthodes, la classe B les possédera aussi car elle les **hérite** de A.

La classe **A** peut implémenter une méthode **afficher**. Si cette méthode n'est pas redéfinie dans **B**, appeler **afficher** sur un objet de type **B** appellera la méthode **afficher** de **A**.

Si on redéfinit **afficher** dans **B**, appeler **afficher** sur un objet de type **B** va utiliser la méthode la plus “spécialisée”, c'est-à-dire celle de **B**.

Mais alors en algo ça sert quand ?

Quand on veut manipuler des structures de données plus complexes que juste des nombres, et surtout quand on cherche à regrouper et gérer des données reliés.

Graphes

Flux de données

Vecteurs

Simulation

Points

Des singes qui font
des maths !?

#AoC11

Les classes dans un graphe

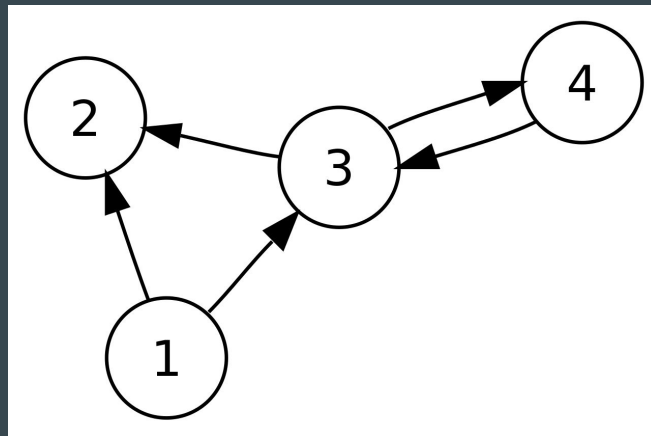
```
class Noeud:
    def __init__(self, nom, voisins = list()):
        self.nom = nom
        self.voisins = set(voisins)

    def ajouter_voisin(self, noeud):
        self.voisins.add(noeud)

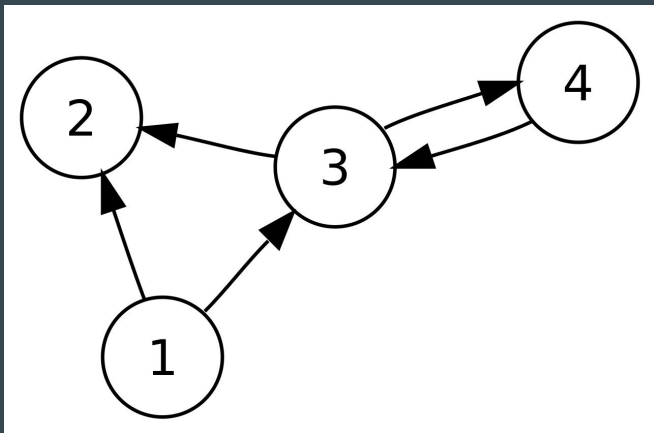
    @property
    def nb_voisins(self):
        return len(self.voisins)

    @staticmethod
    def chemin_entre(noeud1, noeud2):
        chemin = ...
        # algo de Dijkstra
        return chemin
```

On définit une classe Noeud, avec un attribut nom et un attribut voisins, qui est une liste de Noeud



Les classes dans un graphe



```
n1 = Noeud(1)
n2 = Noeud(2)
n3 = Noeud(3)
n4 = Noeud(4, [n3])
n1.ajouter_voisin(n2)
n1.ajouter_voisin(n3)
n3.ajouter_voisin(n2)
n3.ajouter_voisin(n4)
a = n3.nb_voisins # 2
Noeud.chemin_entre(n1, n4)
# retourne [n1, n3, n4]
```


Les limites de la POO

1. Inflexibilité de l'héritage

- L'utilisation excessive de l'héritage peut conduire à des hiérarchies profondément imbriquées et à un **couplage** étroit.
- Les sous-classes peuvent se retrouver avec des relations forcées ou maladroites avec la classe mère.

2. Le problème de la classe de base fragile

- Les modifications apportées à une classe de base peuvent **briser involontairement des sous-classes**.
- Cela oblige à une coordination minutieuse et peut limiter l'évolution indépendante des classes.

3. Violation des principes SOLID

En particulier le **principe de substitution de Liskov (LSP)**, qui stipule que les objets d'une superclasse doivent pouvoir être remplacés par des objets de ses sous-classes sans casser le système.

4. Une trop grande importance accordée aux objets

Peut conduire à une tendance à « objectiver » chaque concept, même lorsque des approches procédurales ou fonctionnelles pourraient être plus simples ou plus appropriées.

5. Difficulté avec la concomitance

Les abstractions de la POO ne s'alignent pas toujours bien sur les modèles de programmation parallèle et concurrente.